

Assignment 1

In this assignment, you will implement a complete **information retrieval system** capable of **indexing** and **searching** a given document collection. This project will give you hands-on experience with core IR concepts, including document parsing, tokenization, indexing algorithms, and search ranking models.

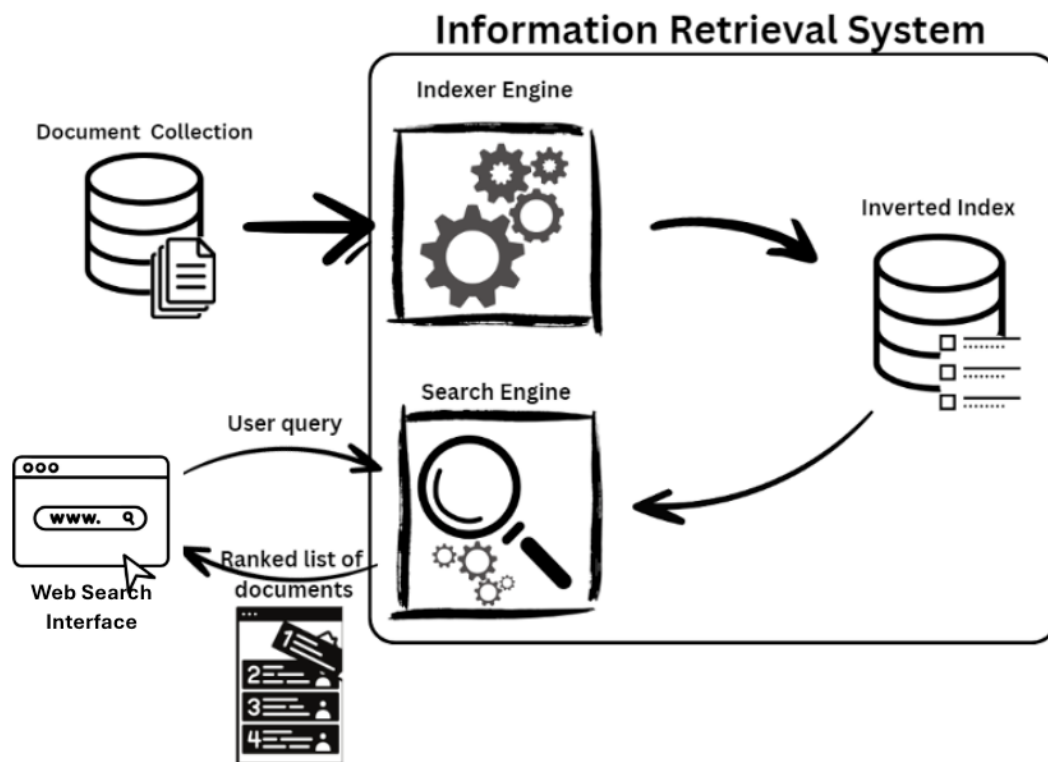


Figure 1: Overview of the components in an Information Retrieval System.

Figure 1 provides an overview of the major components of an Information Retrieval (IR) system. For this assignment, you will implement an indexer and a search engine from scratch, following the requirements listed below.

1. Document collection

You will work with a compiled version of the dump of the Portuguese Wikipedia pages as your corpus.

Corpus (available here "[ptwiki-articles-with-redirects.arrow](#)"):

- Approx. 2.7M documents.
- Each row in this corpus corresponds to a Wikipedia page with the following fields:
 - Title: The title of the page. It can be used to view the page online at <https://pt.wikipedia.org/wiki/{title}>.
 - Text: corpus of the page.
 - Out links: A list of links to other pages that this page contains.
 - Redirect: Indicates whether the page is a redirect to another page.

- **NOTE:** Ensure you perform a corpus clean-up to index only the relevant documents.

2. Indexer Engine

The indexer engine is responsible for parsing the document collection and building an inverted index. Your task is to implement the Single-Pass In-Memory Indexing (**SPIMI**) algorithm to create an inverted index for the provided collection. The index should be a **positional index**, where the posting list includes the term position in the corresponding document. After indexing is complete, all index structures (dictionary, posting lists, document mapping, and other necessary information like metadata) should be **saved on disk** so the Searcher can read them.

- **Memory constraint:** The Indexer implementations need to be able to build the inverted index in less than 2GB of memory. The cli.py includes a memory monitor that will terminate the program if it exceeds 2GB of memory.
- **Tokenizer:** Your tokenizer must be flexible and allow for a customizable configuration. It must support at least the following features:
 - Portuguese Stemmer (you can use nltk)
 - Stop Words
 - Minimum token length
 - You are free to add more transformation options that seems suitable.
- **SPIMI:** Your SPIMI implementation should allow specifying a **minimum term frequency**. Note that this step is performed at index merging time. Example:
 - Let's say the minimum term frequency is 3. This means all terms with a frequency of less than 3 should be ignored.

3. Searcher Engine

The searcher engine is where your system will demonstrate its retrieval capabilities. Develop a search engine that can **load the created index** from disk and provide ranking functionality for user queries. Note that your query processing must use the same tokenization rules as the indexer to ensure consistency. **Tip:** Take advantage of the index metadata to store the tokenizer configuration that was used to create that index.

- **Ranking Algorithm:** Implement the BM25 ranking algorithm and allow for parameterization of k1 and b parameters (assume by default, the values k1=1,2; b=0,75).
- **Relevance Feedback:** Allow the user to search for documents that are like a specific document signalled by the user. (Similar to the google images search functionality)
- **Web Interface:** You are expected to create a web interface for interacting with your search engine. This can be implemented with static pages. One of the criteria for evaluation will be how effectively you display information and enable user interaction, as both are crucial factors in an IR system.

- **Performance:** The user must receive search results in real-time (i.e., in less than one second).
- **Memory constraint:** Here we do not apply the 2GB memory constraint. You are free to load the entire index to memory.

4. Project structure

A skeleton repository is provided with the main entry points required to interact with the indexer and the searcher.

- **The Indexer** entry point is a command-line interface (CLI) used to build and save an inverted index to disk.
- **The Searcher** entry point is a REST API with routes that expose the search functionality through endpoints. Additionally, students are expected to provide a simple web interface to interact with the REST API.

5. Submission Requirements

Your submission should include:

- README - a small report that details all the components and decisions. For instance:
 1. How are the document ingested? (Do you respect the 2GB constraint?)
 2. Characterization of the tokenizer?
 3. How is the index saved what format it follows?
 4. Etc...
- All developed code must be in a runnable state. (We must be able to run your code without issues). **Please verify that your code runs by cloning and configuring everything from scratch before the deadline.**

6. Evaluation Criteria

Your assignment will be evaluated based on the following criteria:

- Work progression along the time
- Presentations
- Information retrieval components
 1. Correctness and completeness of implementation
 1. SPIMI
 2. Searcher
 3. Searcher with relevance feedback
 2. Final ranking results.
- Software engineering components
 1. Quality of the code, organization, design patterns and structures
 2. Overall computational efficiency

```

{
  "title": "Bico (Amares)",
  "text": "Bico \u00e9 uma povoa\u00e7\u00e3o portuguesa sede da Freguesia de Bico do Munic\u00edpio de Amares, freguesia com 2,29\u00a0km\u00b2 de \u00e1rea e 816 habitantes (censo de 2021), tendo, por isso, uma densidade populacional de .\n\n Hist\u00f3ria \nIntegrava o concelho de Entre Homem e C\u00e9lvado, extinto em 31 de dezembro de 1853, data em que passou para o concelho (atual munic\u00edpio) de Amares.ARCHEEVO, PAR\u00d3QUIA DE BICO, [http://pesquisa.adb.uminho.pt/details?id=1002610&ht=PAROQUIA%20DE%20BICO. Acedido a 14 de abril de 2015.\n\nDemografia\t\nA popula\u00e7\u00e3o registada nos censos foi:\n\n Distribui\u00e7\u00e3o da Popula\u00e7\u00e3o por Grupos Et\u00e9rios Ano\t 0-14 Anos\t 15-24 Anos\t 25-64 Anos\t > 65 Anos\t2001 128 92 254 542011 179 90 436 722021 160 104 450 102\n\nCategoria:Freguesias de Amares\nCategoria:Antigas freguesias de Entre Homem e C\u00e9lvado",
  "out_links": [
    "Entre Homem e C\u00e9lvado",
    "Categoria:Freguesias de Amares",
    "C\u00e9lvado (sub-regi\u00e3o)",
    "Portugal",
    "Categoria:Antigas freguesias de Entre Homem e C\u00e9lvado",
    "XVI Recenseamento Geral da Popula\u00e7\u00e3o de Portugal",
    "Amares",
    "freguesia",
    "Munic\u00edpio#Portugal",
    "Regi\u00e3o do Norte",
    "densidade populacional",
    "Vicente de Sarago\u00e7a"
  ],
  "redirect": false
},
{
  "title": "Caldelas",
  "text": "Caldelas pode referir-se a:\nPortugal\nCaldelas (Amares) \u2014 freguesia do concelho de Amares, Portugal\nCaldelas (Guimarães) \u2014 freguesia do concelho de Guimarães, Portugal\nCaldelas (Leiria) - aldeia do concelho de Leiria, Portugal\n\nEspanha\nCastro Caldelas - munic\u00edpio na prov\u00edncia de Pontevedra, comunidade aut\u00f3noma de Galiza, Espanha\nPonte Caldelas - munic\u00edpio na prov\u00edncia de Pontevedra, comunidade aut\u00f3noma de Galiza, Espanha\nComarca da Terra de Caldelas - comarca na prov\u00edncia de Ourense, comunidade aut\u00f3noma de Galiza, Espanha\nCaldelas de Tui - par\u00f3quia no concelho de Tui, prov\u00edncia de Pontevedra, comunidade aut\u00f3noma de Galiza, Espanha\n\nCategoria:Desambiguação\u00e7\u00f5es de top\u00f4nimos",
  "out_links": [
    "Categoria:Desambiguação\u00e7\u00f5es de top\u00f4nimos",
    "Comarca da Terra de Caldelas",
    "Castro Caldelas",
    "Caldelas de Tui",
    "Caldelas (Leiria)",
    "Caldelas (Guimarães)",
    "Ponte Caldelas",
    "Caldelas (Amares)"
  ],
  "redirect": false
}

```